

0P import torch

在jupyter notebook中，如果你在 `import torch` 时一直失败，在安装了所有库之后，且在.py文件中的错误也已经排除后，可能是ipykernel设置的问题。如果你使用的是vscode，其存在自动检查步骤，可以一键安装。

1 Optimization

optimization（优化） 是机器学习中的核心概念，旨在通过调整模型参数来最小化或最大化某个目标函数（通常是损失函数）。优化算法帮助我们找到最佳参数，从而提高模型的性能和预测能力。

Images as points

我们将每个图片转化成平面 $\mathbb{R}^{3WH}$ 上的一个点。这个转化过程会有特定的算法完成，但是总之我们可以对转化后的图片进行分类。我们可以人工地为每一张原始图片打上标签，而若是图片的转化规则是有效的，理论上越靠近具有相同标签的另一张图片，其也具有相同标签的可能性更高。而这个靠近就是关键：

1. L1 距离，曼哈顿距离，Manhattan Distance。是两个向量之间每个维度差的绝对值之和。也就是横纵距离。
2. L2 距离，欧几里得距离，Euclidean Distance。两个向量之间每个维度的平方差之和再开根号。也就是直线距离。

在分类时我们可以考虑以训练集中最接近的图片的点作为某个点的训练结果。又或者，我们可以使用**kNN**，这里的k表示k-Nearest Neighbors，也就是也可以使用前k个距离接近的训练素材来综合决定。

Normal equation

线性回归 (Linear Regression) 是最常见的线性模型，用于根据已有数据预测目标值。如果假设某个目标值和其余一些测量值有线性关系，那么就可以构建一个加权模型：

$$y = \mathbf{w}^T \mathbf{x}$$

其中， $\mathbf{x}$ 是输入特征向量， $\mathbf{w}$ 是权重向量， y 是预测的目标值。通过调整权重 $\mathbf{w}$ ，可以使模型更好地拟合训练数据，从而提高预测的准确性。我们知道，一个 $\mathbf{w}^T$ 实际上就是一个接受向量并输出一个标量的线性映射。其对应了一个超平面，**Loss function**的计算和直线时是一样的，是数据点到超平面的竖直（ y 方向）距离的平方和。

$$L(\mathbf{w}) = ||X\mathbf{w} - \mathbf{y}||^2 = \epsilon^T \epsilon$$

接下来的问题是如何计算权重 $\mathbf{w}$ 。计算可以得到
(本笔记致力于把不好理解的偏微分论证跳过)：

$$\mathbf{X}^T \mathbf{y} = \mathbf{X}^T \mathbf{X} \mathbf{w}$$

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

它又被称为**正规方程 (Normal Equation)**。

你会发现其在Python中的实现其实非常简单：

```
def nsolve(X,y):
    """
    其实一行就可以哦~
    Arguments:
    X : Data matrix
    y : Target values
    Returns:
    w : Weights
    """
    XtX = X.T.dot(X)
    Xty = X.T.dot(y)
    XtX_inv = np.linalg.inv(XtX)
    w = XtX_inv.dot(Xty)
    return w
```

与此同时，生活中的很多问题不一定是线性的，那么我们可以使用**多项式回归 (Polynomial Regression)**。我们可以对比一下结果

$$\begin{aligned} \mathbf{y} &= \mathbf{X}\mathbf{w} + \epsilon \\ L(\mathbf{w}) &= ||X\mathbf{w} - \mathbf{y}||^2 = \epsilon^T \epsilon \\ \mathbf{w} &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \\ y &= \Phi \mathbf{w} + \epsilon \\ L(\mathbf{w}) &= \epsilon^T \epsilon \\ \mathbf{w} &= (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y} \end{aligned}$$

其中

$$\Phi = \begin{bmatrix} \phi(\mathbf{x}^1)^T \\ \phi(\mathbf{x}^2)^T \\ \vdots \\ \phi(\mathbf{x}^N)^T \end{bmatrix}$$

这里我们就能发现**Loss function**的作用了，它能帮助我们衡量模型的好坏，从而指导我们调整模型参数以提高预测性能。

Regularization

作为一种防止模型过拟合（overfitting）的策略，**正则化 (Regularization)** 通过在损失函数中添加一个惩罚项来限制模型的复杂度。

$$L(\mathbf{w}) = ||X\mathbf{w} - \mathbf{y}||^2 + \lambda R(\mathbf{w})$$

这个额外的项就可以使得权重不会过大，从而防止过拟合。

2 Neural Networks

Stochastic Gradient Descent (SGD)

在机器学习中，我们通常会使用**梯度下降法 (Gradient Descent)** 来优化模型参数，以最小化损失函数。相对于纯粹随机的选点，你可以想象我们利用了超平面的斜率来决定下一个点的位置，从而更快地接近最优解。

$$\mathbf{w}_{t+1} := \mathbf{w}_t - \alpha \nabla f(\mathbf{w}_t)$$

那么在最低点附近，可能会出现震荡的情况；或者可能收敛缓慢。为了解决这个问题，我们可以引入**动量 (Momentum)** 的概念。动量帮助我们在更新参数时考虑之前的更新方向，从而减少震荡并加速收敛，并且避免了陷入局部最优。它的特点是there is an adaptive learning rate，也就是有一个自适应的学习率。

$$\mathbf{v}_{t+1} := \beta \mathbf{v}_t + \nabla f(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} := \mathbf{w}_t - \alpha \mathbf{v}_{t+1}$$

Adam (Adaptive Moment Estimation) 则结合了动量和自适应学习率的思想。它通过计算梯度的一阶矩和二阶矩，并且对它们进行偏差校正，从而实现更稳定和高效的参数更新。这个偏差值校正指的是在初始阶段，由于初值为0，两个矩的数值很小，所以需要进行校正，加强学习率。

$$\begin{aligned} g_t &= \nabla f(\mathbf{w}_t) \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \mathbf{w}_{t+1} &= \mathbf{w}_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned}$$

尤其是，注意到， $t = 1$ 时， $m_t = 0 + (1 - \beta_1)g_1$ 和 $\hat{m}_t = \frac{(1-\beta_1)g_1}{1-\beta_1^1} = g_1$ 恰好消除了初始学习率不足的问题。

Activation Functions

在神经网络中，**激活函数 (Activation Functions)** 用于引入非线性，使得神经网络能够学习复杂的模式和关系。常见的激活函数包括：

1. **Sigmoid 函数**：将输入映射到 (0, 1) 之间，适用于二分类问题，但容易导致梯度消失。

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

2. **ReLU (Rectified Linear Unit) 函数**：将负值映射为0，正值保持不变，计算简单且能有效缓解梯度消失问题。

$$\text{ReLU}(x) = \max(0, x)$$

3. **Tanh 函数**：将输入映射到 (-1, 1) 之间，中心对称，适用于隐藏层，但同样存在梯度消失问题。

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

4. **Leaky ReLU 函数**：对负值引入一个小的斜率，避免了ReLU的“死亡”问题。

$$\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

我们刚刚提到使用激活函数可以引入非线性，那么为什么非线性这么重要呢？
一个重要的反例就是**XOR problem**。也就是两类问题在平面上的分布相互交叉，那线性模型是无法区分的。但是具有非线性特征的神经网络就可以做到。
之所以使用特定的激活函数，是因为对并行计算来说他们是相对容易的，对于机器学习使用的计算平台而言效率更高。不过这个似乎不是这门课的重点，可能是默认了这是常识。

MLP (Multilayer Perceptron)

MLP (Multilayer Perceptron, 多层感知器) 将多层的矩阵乘法和非线性激活函数堆叠，构成有**隐藏层 (Hidden layer)** 的前馈网络。
在**MLP** 中，**反向传播 (Backpropagation)** 是一种用于计算梯度的算法，它通过链式法则将误差从输出层传递回输入层，从而更新网络中的权重。这里我们介绍加和乘两种基本操作的反向传播过程。

1. **加法节点 (Addition Node):**
 - 前向传播： $z = x + y$
 - 反向传播： $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot 1, \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \cdot 1$
2. **乘法节点 (Multiplication Node):**
 - 前向传播： $z = x \cdot y$
 - 反向传播： $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot y, \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \cdot x$

我们举一个例子：

$$\begin{aligned} x &= 3, y = 5, z = -2 \\ q &= x + y = 8 \\ f &= q * z = -16 \end{aligned}$$

然后我们假设损失梯度为1：

$$\begin{aligned} \frac{\partial L}{\partial f} &= 1 \\ \frac{\partial L}{\partial z} &= \frac{\partial L}{\partial f} * q = 1 * 8 = 8 \\ \frac{\partial L}{\partial q} &= \frac{\partial L}{\partial f} * z = 1 * (-2) = -2 \\ \frac{\partial L}{\partial x} &= \frac{\partial L}{\partial q} * 1 = -2 * 1 = -2 \\ \frac{\partial L}{\partial y} &= \frac{\partial L}{\partial q} * 1 = -2 * 1 = -2 \end{aligned}$$

Positional Encoding

这个概念强调了在神经网络中引入位置信息，在LLM中可以简单的被理解为不同的token的位置信息可以被嵌入到编码之中。对于Visual Computing，我们可以通过将像素的坐标信息编码为高维向量，并将其与像素值一起输入神经网络，从而使网络能够更好地理解图像的空间结构和关系。这个概念之后还会提到。

3 Convolutional Neural Networks (CNNs)

卷积，将两个函数结合的运算， $((f * g)[n] = \sum f[m] \cdot g[n - m])$ 。图像是矩阵，所以图像是函数，我们一会就讲的卷积核也是矩阵，所以也是函数。那么它们就可以进行卷积运算，得到一个新的矩阵，也就是新的图像。

Gaussian Smoothing

在图像和音频处理时，可能会需要去除噪声。虽然我们可以使用简单的区域平均（均值滤波）来实现这一点，但更常用的是**高斯平滑 (Gaussian Smoothing)**。高斯平滑使用高斯函数作为权重，对邻近像素进行加权平均，从而更有效地保留图像的细节。高斯核的公式为：

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$$

其中， σ 是标准差，决定了平滑的程度。较大的 σ 会导致更强的平滑效果，但可能会模糊图像细节。 x 和 y 决定了高斯核的范围。例如，下面定义了一个5x5的高斯核生成函数：

```
def gkern(sig=1.):
    ax = np.arange(-2, 3) # [-2, -1, 0, 1, 2]
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-(xx**2 + yy**2) / (2.0 * sig**2))
    # 因为要归一化，所以前面的2pi sig^2可以省略
    out_kernel = kernel / np.sum(kernel)

H, W = img.shape
kH, kW = kernel.shape
# 省略数据导入和初始化，以下是卷积操作的核心部分
for i in range(H):
    for j in range(W):
        region = padded[i:i+kH, j:j+kW]
        output[i, j] = np.sum(region * kernel)
```

当然在实际引用中，我们不止需要平滑过滤器，比如锐化（Sharpening），通过将中心点周围的核值设为负值来增强图像的边缘细节。比如这样：

$$S(x,y)=\begin{bmatrix}0&-1&0\\-1&5&-1\\0&-1&0\end{bmatrix}$$

MLP vs CNN

我们之前已经提到过**MLP（Multilayer Perceptron）**，和CNN一样，训练好的模型可以表示某种映射关系。例如一个记录了某张图片信息的模型可以将位置数据 $[x,y]$ 映射到像素值 $[r,g,b]$ 上。但是，假如我们有一个节点，接受一个图像的所有像素值作为输入，并输出另一个图像的所有像素值。那么这个节点实际上就是一个非常大的矩阵乘法运算。而卷积可以视为一个后续节点只接受局部区域的输入，从而大大减少了参数数量。如下所示，如果我们的卷积核是一个 5×5 的矩阵，那实际上也就用到了 w_0 到 w_{24} 这25个权重，因为卷积的特性，斜线上的权重可以是相同的。而如果是一个全连接的MLP节点，那么它需要使用整个图像的所有像素值作为输入，假设图像大小为 $K \times K$ ，那么这个节点就需要使用 K^4 个权重。

我们就不打那么多了

$$\begin{bmatrix}y_1\\y_2\\y_3\\y_4\\\vdots\\y_K\end{bmatrix}=\begin{bmatrix}w_0&w_1&w_2&0&0&\dots&0\\0&w_0&w_1&w_2&0&\dots&0\\0&0&w_0&w_1&w_2&\dots&0\\0&0&0&w_0&w_1&\dots&0\\\vdots&\vdots&\vdots&\vdots&\vdots&\ddots&\vdots\\0&0&0&0&0&\dots&w_0\end{bmatrix}\cdot\begin{bmatrix}x_1\\x_2\\x_3\\x_4\\\vdots\\x_K\end{bmatrix}$$

所以，MLP和CNN的区别就在于，CNN的每个节点只接受局部区域的输入，从而大大减少了参数数量。

Padding

在卷积操作中，**填充 (Padding)** 是指在输入图像的边缘添加额外的像素，以控制输出图像的尺寸和保留边缘信息。常见的填充方式包括零填充（Zero Padding），即在图像边缘添加值为零的像素。

例如对一个边长为 W 的图像进行 P 像素的填充后，新的边长将变为 $W + 2P$ 。经过卷积核大小为 K 的卷积操作后，输出图像的边长将变为 $W + 2P - K + 1$ 。

Pooling

在卷积神经网络中，**池化 (Pooling)** 是一种下采样技术，用于减少特征图的尺寸，从而降低计算复杂度和防止过拟合。常见的池化方法包括最大池化（Max Pooling）和平均池化（Average Pooling）。包括**最大池化 (Max Pooling)**、**平均池化 (Average Pooling)** 等，都是合理可行的。

值得注意的是，如果我们先卷积然后池化，然后把图像对称，和先对称然后卷积、池化，结果是一样的。

UnPooling 则是池化的逆过程，用于将下采样后的特征图恢复到原始尺寸。常见的方法包括最近邻插值（Nearest Neighbor Interpolation）和双线性插值（Bilinear Interpolation）。不过需要注意的是，池化操作会丢失一些信息，因此反池化无法完全恢复原始特征图。

以上是两种 **下采样 (Downsampling)** 和 **上采样 (Upsampling)** 的方式。当然，卷积自然也是一种下采样，而上采样自然还包含反卷积（Transposed Convolution）。

Strided Convolution

通过增加卷积的步长，可以降低输出的尺寸。例如，假设我们有一个输入图像，其边长为 W ，并且我们使用一个大小为 K 的卷积核。如果我们设置步长为 S ，那么输出图像的边长是 $\frac{W-K}{S} + 1$ 。通过调整步长，我们可以控制输出图像的尺寸，从而实现下采样的效果。

Normalization

Normalization（归一化） 是一种数据预处理技术。包括**Whitening**和**Batch Normalization（BN）**。这里跳过其数学细节，只要记住它们都能对均值（mean，变成0）和标准差（standard deviation，变成1）进行归一化即可。

其中的好处包括减少了对初始权重的敏感性，并使训练过程更加稳定。

Residual Network

在讲解这个之前，我们先解释一下**Naïve solution** 和 **Plain block**。其中前者和我们在MLP那一节最开始用的解释很像，通过连续的Conv-ReLU堆叠来构建网络。而后者则是在每个Conv-ReLU后面加上一个卷积层。

而**Residual Network（残差网络）** 则引入了**跳跃连接 (Skip Connections)** 的概念，通过将输入直接添加到输出，从而形成所谓的“残差块 (Residual Block)”。例如，通过跳过一些层（设其效果为 $F(x)$ ），直接将输入 x 添加到输出中，形成 $y = F(x) + x$ 。

Fully Convolutional Network (FCNN) 和 U-Net（Hourglass Network）

一个**全卷积网络 (Fully Convolutional Network, FCNN)** 不包含MLP。它的输入和输出的尺寸可能是相似的，于是我们可以通过神经网络对图像进行像素级别的处理。

而经过了卷积等操作运算过的图像细节会永久地丢失，使用上一节提到的skip connection可以在一定程度上缓解这个问题。

U-Net（Hourglass Network） 则一定具有相同的输入输出尺寸。通过在对称的编码器-解码器结构中使用skip connection，也可以有效地保留图像细节信息。

例如，在物体识别中，现在的技术可以判断图片中某个区域中是否有可识别物体，甚至可以判断每个像素是这个物体的概率。这就是FCNN和U-Net的应用。

注意，未来的章节有一个叫autoencoder的概念，用于（可能）无监督的图片重建。而U-Net用于像素级分类，两者虽然结构相似但是不同概念。

这个区分像素属于什么物体的任务，叫做**语义分割 (Semantic Segmentation)**。

Dropout

通过在训练过程中随机“丢弃”一部分神经元。这个比例可以自己设定，比如在某一层中是50%。这样，这一层剩余神经元的输出就要乘以2，以保持整体输出的期望值不变。这样的操作可以让网络更有鲁棒性（robustness），防止过拟合。

Softmax function

Softmax 函数 将输出转换为概率分布。它将每个输入映射到 (0, 1) 之间，并确保所有类别的概率和为1。其公式为：

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

不要问为什么又 e 又 z 又 i 的，和复数域没有一点关系。还是个把向量归一化（和变为1）的操作

Fast R-CNN

这是一种通过在输入后直接套一个卷积神经网络先给出区域建议（Region Proposal）然后再在各个小区域中应用第二层网络来确定分类和边界信息。

3P Denoising CNN model

我们来看一个简单的去噪神经网络模型：

```

class DenoisingDB(Dataset):
    def __init__(self, input_imgs_path, cleaned_imgs_path):
        pass # 略

class DenoisingCNN(nn.Module):
    def __init__(self):
        super(DenoisingCNN, self).__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.1, inplace=True),

            nn.Conv2d(64, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.1, inplace=True),

            nn.MaxPool2d(2),

            # 后续层略
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 2, stride=2),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.1, inplace=True),

            # 后续层略
        )
        self.res_scale = 0.1 # Residual scaling factor

    def forward(self, x):
        identity = x
        out = self.encoder(x)
        out = self.decoder(out)
        x = identity + self.res_scale * out
        return x

def loss_function(prediction, target):
    return F.mse_loss(prediction, target)

denoiser = DenoisingCNN().to(device)
optimizer = torch.optim.Adam(denoiser.parameters(), lr=1e-3)
# 其余数据加载略

for epoch in range(10):
    denoiser.train()
    for noisy, clean in train_loader:
        optimizer.zero_grad() # 清空上一轮的梯度
        output = denoiser(noisy) # 等于 denoiser.forward(noisy)
        loss = loss_function(output, clean)
        loss.backward() # torch自动计算梯度
        optimizer.step() # 使用adam更新参数，知识点在第二章哦

```

这里其实没有什么很难理解的结构，最后的主程序可以看到就是我们之前讲过的训练神经网络的标准流程。

一个小细节是**残差连接 (Residual Connection)** 的使用。 `x = identity + self.res_scale * out` 这一行代码实现了输入和输出的加权和，从而帮助网络更好地学习去噪映射。

4 Visualizing and Understanding Neural Models

Visualizing Activations

如果一个神经网络的权重已经固定了,也就是weights are fixed，我们说它是frozen的。

那么，现在我们确定神经网络的结构（architecture）和权重（weights）后，我们开始研究输入，将原本确定的样板作为未知数来考虑。其中一个比较常见的探索思路是，我们希望找到一个输入，使得某个神经元的激活值（activation）最大化。

Saliency

通过覆盖（Occlusion），我们可以观察到不同区域对于各神经元输出的影响，从而确定同一张图片的哪一个区域具有**显著性（saliency）**。

Guided Backpropagation 和 Grad-CAM

- Guided Backpropagation** 使用我们之前介绍过的反向传播（Backpropagation）的计算方式。首先我们指定输入，也就是我们想要知道重点在哪的图片，然后我们指定感兴趣的分类，也就是结尾处的一个输出节点，当然也可以是中间任意一个神经元。我们计算从这个节点开始梯度不为零（同层其余设为0）推导出的输入层的梯度。
将这个梯度以图像的形式展示出来，我们就能看到哪些输入像素对这个输出节点的激活值贡献最大。一般来说就是输入图案的轮廓。
在卷积层中它其实进行了改变，在ReLU的反向传播中只保留正梯度。但是课件忽略了这点，大概不重要吧。
- Grad-CAM（Gradient-weighted Class Activation Mapping）** 则是通过计算卷积层输出特征图（feature map）对目标类别的梯度，来生成热力图（heatmap）。你可能注意到了这里的结果显示在卷积层的最后，这是因为卷积层具有原图的空间信息，所以这张热力图已经可以放大到原图大小了。
注意，为了更高精度的边缘，Guided Backpropagation需要计算到输入层
- Guided Grad-CAM** 将上述两种方法的结果相乘，从而结合了两者的优点。

Gradient Ascent

Gradient Ascent（梯度上升） 是一种通过梯度信息不断调整输入，使目标函数的输出值尽可能大的方法。

这可以被应用到生成**对抗样本（Adversarial Examples）**中。通过对输入图像进行微小的扰动，使得神经网络产生错误的分类结果。如此一来，我们可以测试神经网络的鲁棒性，并改进其防御机制。

Feature Inversion（特征反演） 则可以能使某个特定神经元给出相同输出的图像。

DeepDream（深度梦境） 需要指定一个卷积层，然后通过梯度上升不断调整输入图像，使得该层的激活值最大化。这样，我们可以生成具有梦幻般视觉效果效果的图像，展示神经网络对特定特征的敏感性。

Autoencoders

它指的是由编码器（Encoder）和解码器（Decoder）组成的神经网络结构。编码结果是一个维数（Dimensionality）较低的数据。但是它尽可能地有效保留了输入信息。这样使用解码器可以使结果非常接近原始图片。

$$Image \xrightarrow{\text{encoder}} Z \xrightarrow{\text{decoder}} Image'$$

之前我们讲过的U-Net和FCNN其实也是autoencoder的一种变体，只不过它们的目标是像素级分类，而不是重建图像。

Texture Synthesis

我们学习一个给定的具有纹样的图片，同时输入一个随机噪声图片。然后就可以像拼图一样，根据周围像素信息计算出新的一像素最适合的颜色。以下是一个例子：

$$S = \begin{bmatrix} w_0 & w_1 & w_2 & w_3 & w_4 \\ w_5 & w_6 & w_7 & w_8 & w_9 \\ w_{10} & w_{11} & \text{待确定格} & 0 & 0 \end{bmatrix}$$

用更加神经网络的方式需要以下几个概念：

VGG 是一种层数较多的卷积神经网络架构，当然它做的事情和上一章讲的是一样的，在提到VGG时，它的输出一定是分类结果。

Gram Matrix 是将某一层的特征图（feature map）进行矩阵乘法得到的结果。假设某一层的特征图大小为 $C \times H \times W$ ，其中 C 是通道数， H 和 W 分别是高度和宽度。那么其Gram Matrix就是一个 $C \times C$ 的矩阵，每个数据表示每两个通道之间的内积。

在VGG中输入我们想要学习的纹理图片，然后计算出每一层的Gram Matrix。接着我们输入一个随机噪声图片，然后通过梯度下降不断调整这个噪声图片，使得它在各层的Gram Matrix尽可能接近纹理图片的Gram Matrix。这个差自然就产生了各层的梯度，这个梯度可以backpropagation到输入层，从而使用gradient descent调整输入图片。

同理，要将某张图的纹理风格应用到另一张图上，只需要将前者作为上一段的纹理图片，后者作为初始输入图片即可。

Feature Loss

相较于逐像素地比较两张图像的差异，**特征损失 (Feature Loss)** 则是通过比较两张图像在高层语义特征空间中的差异来衡量它们的相似性。

1. SSIM (Structural Similarity Index, 结构相似性):

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + a)(2\sigma_{xy} + c)}{(\mu_x^2 + \mu_y^2 + a)(\sigma_x^2 + \sigma_y^2 + c)}$$

通过比较局部亮度、对比度和结构信息，提供了一种更符合人类视觉感知的图像质量评估方法。

但是它不适合用在神经网络的训练中，因为它不可微分。同时对细节的敏感度不够。

2. LPIPS (Learned Perceptual Image Patch Similarity):

$$\text{LPIPS}(x, y) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (f_l(x)_{h,w} - f_l(y)_{h,w})\|_2^2$$

通过预训练在神经网络中各层参数的差对应的权重，对神经网络内读取到的特征进行加权比较。

它比较了深层特征的差异 (distance between deep features)，并通过学习权重来更好地捕捉人类感知的相似性。

4P Loss of Images

我们来看这段程序，其被用于计算两张图片的差异

```
def _rgb_feature(image):
    return image.reshape(-1).astype(np.float32)

def _hist_feature(image):
    feats = []
    for c in range(3):
        h, _ = np.histogram(image[..., c], bins=32, range=(0.0, 1.0))
        h = h / (h.sum())
        feats.append(h)
    return np.concatenate(feats).astype(np.float32)

# vgg_full = models.vgg19(weights=models.VGG19_Weights.DEFAULT)
# 其余导入和初始化省略

def _vgg_batch_featuremaps(pil_list):
    _ensure_vgg()
    tensors = [ _vgg_transform(img) for img in pil_list ]
    batch = torch.stack(tensors, dim=0).to(device)
    with torch.no_grad(): # no gradient computation
        feats = _vgg_model(batch)
        N, C, H, W = feats.shape
    return feats.view(N, C*H*W).cpu().numpy().astype(np.float32)

def _vgg_batch_gram(pil_list):
    _ensure_vgg()
    tensors = [ _vgg_transform(img) for img in pil_list ]
    batch = torch.stack(tensors, dim=0).to(device)
    with torch.no_grad():
        feats = _vgg_model(batch)
        N, C, H, W = feats.shape
        f = feats.view(N, C, H*W)
        G = torch.bmm(f, f.transpose(1, 2)) / float(C*H*W)
        G_flat = G.view(N, C*C).cpu().numpy().astype(np.float32)
    return G_flat

diffs = methodName(ImageA) - methodName(ImageB)
dists = np.linalg.norm(diffs, axis=1)
```

最后的 `np.linalg.norm` 会计算两个向量的L2距离。也就是说我们需要将图片处理为任意维度的向量，然后计算它们的差异。

在接下来的讲述中，假设输入的图片是一个 224 x 224 x 3 的numpy数组。

RGB

`reshape(-1)` 的含义是不论输入格式地扁平化，也就是说其是一个长度为 $224 \times 224 \times 3 = 150528$ 的向量。

Histogram

对于三个通道，依次计算其数值的直方图，这里使用了32个bin，也就是将像素值分为32个区间，统计每个区间内的像素数量。在归一化后，将三个通道的结果合并为一个 $32 \times 3 = 96$ 维的向量。

VGG Feature

首先我们对图片进行预处理，使其符合VGG网络的输入要求：

```
_vgg_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])
```

这是也就是图片被转换为 $3 \times 224 \times 224$ 的张量。`ToTensor` 会将其转换为 $[0,1]$ 范围内的浮点数。然后进行归一化处理。

```
_vgg_model = vgg_full.features.to(device).eval()
```

然后使用库函数中预训练的VGG19模型，其结果包含形状为 $N \times C \times H \times W$ 的张量，其中 N 是批量大小（输入的图片数量）， C 是通道数， H 和 W 分别是高度和宽度。

`view` 是一个对张量形状进行调整的函数，我们将其调整为 $N \times (C \times H \times W)$ 的形状。也就是说每张图片是一个 $C \times H \times W$ 维的向量。

这种算法得到的是两张图片的 **content loss**，也就是内容差异。

VGG Gram matrix

在此基础上我们求Gram Matrix：

`torch.bmm`是批量矩阵乘法函数。机理如下

```
f                = (N, C, H*W) # 每个通道里的像素在同一个向量里
f.transpose(1, 2) = (N, H*W, C) # dim=0的N不变，另外两个向量交换
result           = (N, C, C)    # 对N张图片的C个通道，每两个通道进行内积
```

所以我们的返回值是 $N \times (C \times C)$ 的张量。也就是说每张图片是一个 $C \times C$ 维的向量。

这种算法得到的是两张图片的 **style loss**，也就是风格差异。例如，两张图片的颜色和纹理等风格相似，但是内容不同，那么它们的style loss会很小，而content loss会很大。

5 Sequential Data (transformers)

在之前，我们的模型主要用于图案分类，虽然我们也提到了一些模型（如U-Net）的输出也是图案，但是依然属于 **one-to-one mapping**。实际上还存在如：

one-to-many mapping，例如图像描述生成（Image Captioning），输入是一张图片，输出是一段描述该图片的文字。

many-to-one mapping，例如视频分类（Video Classification），输入是一段视频，输出是该视频类别。

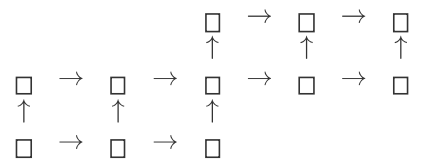
many-to-many mapping，例如动画合成（Animation Synthesis），输入是一段动作序列，输出是另一段动作序列。

RNN (Recurrent Neural Networks)

RNN (Recurrent Neural Networks, 循环神经网络) 通过引入循环连接，使得网络能够处理序列数据。它们在时间步之间共享参数，从而捕捉序列中的时间依赖关系。

```
def RNNStep(oldHidden, newInput):
    NewHidden = function1(oldHidden, newInput)
    NewOutput = function2(newHidden)
    return NewHidden, NewOutput
```

上述代码体现的就是RNN的基本结构。可以理解为：



注意到，我们显然不能为每一个视频都做一个这样的结构。所以这个中间层应该是一个通用的结构。所以在上面的代码中， `RNNStep` 有两个输入和两个输出。

这里我们要提一下**token** 这个概念，与LLM中的一样，它 是被提取的最小语义单元。之前讲到过的autoencoder中的Z，就可以作为token，因为它是低维度的，却可以有效地表达输入信息。但是通常，token确实是一个向量，与像素相比确实具有更多语义。这也就代表了 we 不能再对它使用卷积和激活函数了，而是还是使用MLP。

我们可以将一个图片分块得到很多**patches**，然后将每个patch编码成一个token。

Problems of Gradients

1. **Exploding gradients (梯度爆炸)**：如果梯度可能会变得非常大，会导致权重更新过大，从而使模型不稳定。可以使用**梯度裁剪 (Gradient Clipping)**，即将梯度限制在一个预定的范围内。
2. **Vanishing gradients (梯度消失)**：而如果梯度非常小，权重则几乎不更新。这个没有特定的方法，一会要讲的注意力与其有关。
3. **Truncated gradients (截断梯度)**：对于过长的序列，计算梯度是几乎不可能的，这是可以将其分断。 （这不是这门课的重点）

Long Short-Term Memory (LSTM)

它引入**context** 的概念，来对前文的信息进行选择性地记忆和遗忘。首先根据这一节的输入 x_t ，上一节传递的短期输入 h_{t-1} ，以及权重 W 确认四个数值：

$$\begin{bmatrix} i \\ f \\ o \\ g \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} W \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix}$$

然后，对于context c_{t-1} ：

$$c_t = f \odot c_{t-1} + i \odot g$$

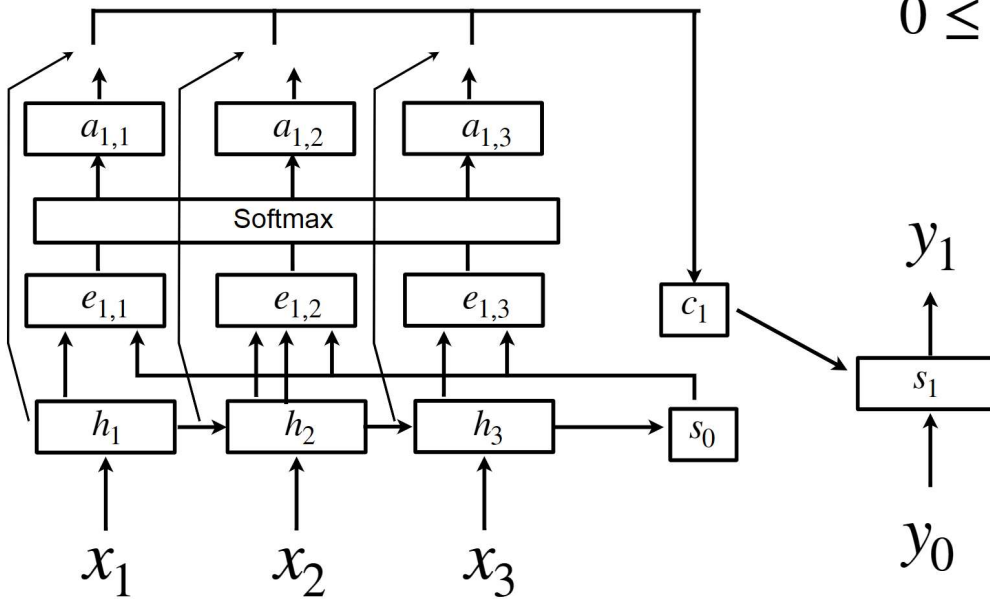
$$h_t = o \odot \tanh(c_t)$$

这里的 f 是遗忘门 (forget gate)，决定 了多少前文信息需要被遗忘； i 是输入门 (input gate)，决定 了多少当前信息需要被记忆； o 是输出门 (output gate)，决定 了多少信息需要被输出。

Attention

Attention (注意力) 与CNN最大的区别在于，它是会**adapt** 的。也就是说，它会根据输入动态地调整权重。

RNN with Attention



$$e_{t,i} \leftarrow g_{att}(s_{t-1}, h_i)$$

$$0 \leq a_{t,i} \leq 1 \quad \sum_i a_{t,i} = 1$$

$$c_t = \sum_i a_{t,i} h_i$$

这张图非常重要请你记住

其中, h 是被处理完后的token, 在使用它算出解码器状态 s 后 (只使用 h 计算 s_0), 我们可以用各个 h_i 和 s 计算出一个注意力分数 e_i 。然后通过 softmax 函数将其归一化为权重 a_i 。

$$e_i = \text{score}(h_i, s)$$

$$a_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}$$

这里的 a_i 就代表了每个 token 对当前解码器状态的重要性。所以要求出上下文向量 (Context Vector) c , 只需要按权重累加:

$$c = \sum_i a_i h_i$$

然后和之前的过程一样, c 和 y_{t-1} 被用于算出 s_t 和输出 y_t 。

现在我们已经得到了 s , 且 h 是不变的, 我们就可以再次回到注意力层计算出新的 c , 然后继续解码。

Self/Cross Attention Layer

之前我们讲的属于 **cross attention**, 也就是编码器和解码器之间的注意力。而 **Self Attention Layer (自注意力层)** 则是指输入和输出是同一组 token 的情况。

Input vector X

Key vector $K = XW_K \in \mathbb{R}^{n_k \times d}$

Value vector $V = XW_V \in \mathbb{R}^{n_k \times d}$

Cross Query vector $Q \in \mathbb{R}^{n_q \times d}$ (即和另外两个矩阵无关的)

Self Query vector $Q = XW_Q \in \mathbb{R}^{n \times d}$ (这种情况下 $n_q = n_k = n$)

Similarity $E = QK^T \rightarrow$ Attention weights $A = \text{softmax}(E) \in \mathbb{R}^{n_q \times n_k}$,

Output $Y = AV \in \mathbb{R}^{n_q \times d}$

也就是说, Q 是一个 token 提出的问题, 而 K 相当于是一个索引, V 则是对应的实质内容。对于 cross attention 来说, 是让一个序列去查询另一个序列的信息。而 self attention 则是让一个序列去查询它自己的信息。

6 VAE and GAN

Normalising Flow

我们可以通过一个简单的分布（如高斯分布）来生成数据，然后通过一个可逆的神经网络将其映射到目标分布上。这个过程称为**Normalising Flow（归一化流）**。

这个架构的有点在于其分布非常明确，且是可逆的，所以既可以被用来生成，也可以被用来推断（inference）。但是它的缺点在于需要设计一个可逆的网络结构，这限制了模型的复杂性。

AE

我们之前已经讲过**Autoencoder（自编码器）**，你会发现虽然它是在一个分类器的后面跟了应该对称的解码器，但是它的两端数据实际上已经可以是无标签的了。

但是它使用的decoder应该被称为generative model的雏形，因为它并没有学习数据的分布。我们在最开始就说过，图片可以被当成低维空间中的一个点。那么在被要求还原一张图片时，autoencoder其实给出的是它觉得可以映射到这个点的图片的累加，而不是最有可能的结果。

Clustering

之前我们讲的都是监督学习（Supervised Learning），也就是有标签的数据。我们输入数据，并将其映射到标签（labels）上。而无监督学习（Unsupervised Learning）则是没有标签的数据。其中比较有代表性的就是生成模型（Generative Models）。

Clustering（聚类）是一种无监督学习方法，用于将数据点分组。这里先以K-means为例。

所有的无标签的数据在图中分布，我们初始一些质心（centroids），然后将每个数据点分配给距离最近的质心。接着，根据被分配到的数据点的分布重新计算每个簇的质心。重复这个过程，直到质心不再发生变化或达到预定的迭代次数。这些质心最终会收敛到某一簇数据点的中心位置。

Expectation Maximization (EM) algorithm

在很早的章节中我们使用高斯核来对图像进行处理，但是有时候需求的处理并不是单个高斯核，而是多个高斯核的混合。这就引出了**高斯混合模型 (Gaussian Mixture Model, GMM)**。

和我们在讲聚类时的算法类似，我们的计算包括两个步骤：

1. **Expectation Step (E-step)**：计算每个数据点属于每个高斯分布的概率。

$$R_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

2. **Maximization Step (M-step)**：根据这些概率重新估计高斯分布的参数（均值、协方差和混合系数）。

$$\begin{aligned}\mu_k &= \frac{\sum_{i=1}^N R_{ik} x_i}{\sum_{i=1}^N R_{ik}} \\ \Sigma_k &= \frac{\sum_{i=1}^N R_{ik} (x_i - \mu_k)(x_i - \mu_k)^T}{\sum_{i=1}^N R_{ik}} \\ \pi_k &= \frac{1}{N} \sum_{i=1}^N R_{ik}\end{aligned}$$

这个运算的收敛结果是数个高斯分布的混合，可以较好地拟合数据的分布。我们给每一个高斯分布一个标签 z ，称为**latent code**。

VAE（Variational Autoencoder）

$$p_{\theta}(\mathbf{x}) = \int_z p_z(z) \mathcal{N}(\mathbf{x}; g_{\theta}^{\mu}(z), g_{\theta}^{\Sigma}(z)) dz$$

上面这个公式中， $\mathcal{N}$ 内部的定义是given z 后，也就是框定一个高斯分布区域之后，图片在向量空间中的分布。而对所有 z 的积分，就是VAE所学习到的图片分布。

但是我们知道在机器学习领域是不会有微积分直接表示的，所以我们使用**ELBO (Evidence Lower Bound)** 来近似这个积分。

$$\log p_{\theta}(\mathbf{x}) \geq \mathbb{E}_{q_{\phi}(z|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|z)] - D_{KL}(q_{\phi}(z|\mathbf{x}) || p_z(z)) = ELBO(q, \theta)$$

也就是说它是那个积分的log的下界。我们希望通过训练使ELBO尽可能大。

对于这个表达式，我们也是可以EM的：

1. **E-step** 优化 q （encoder，通过修改 ϕ ）来最小化 KL：

$$\log p_{\theta}(\mathbf{x}) = ELBO(q, \theta) + D_{KL}(q_{\phi}(z|\mathbf{x})||p_{\theta}(z|\mathbf{x}))$$

2. **M-step** 优化 θ （decoder）来最大化期望 log-likelihood：

$$\theta^{new} = \arg \max_{\theta} ELBO(q, \theta)$$

Vector Quantization-VAE（VQ-VAE）

上面我们的表示中可以对 z 积分就是因为 z 是连续的变量。而**VQ-VAE（Vector Quantization-VAE）** 则是根据codebook将 z 离散化为有限个类别。也就是说，我们将潜在空间划分为若干个区域，每个区域对应一个离散的向量（codebook entry）。在编码过程中，输入数据被映射到最近的向量，从而实现离散化。

GAN (Generative Adversarial Networks)

GAN（Generative Adversarial Networks，生成对抗网络） 通过引入一个判别器（Discriminator）来对抗生成器（Generator）。生成器试图生成逼真的数据，而判别器则试图区分真实数据和生成数据。

Semantic control with GANs

在latent space中会包含一些语义信息，例如在生成的人脸图像中，某些方向可能对应于“微笑”或“戴眼镜”等特征。通过在latent space中沿着这些方向进行操作，可以实现对生成图像的语义控制。例如，通过调整latent vector，可以让生成的人脸图像从不戴眼镜变为戴眼镜，或者从不微笑变为微笑。这些语义方向通常通过 PCA、GANSpace、SeFa 等方法从 latent space 或 generator weights 中自动发现。

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

VQ-GAN

这个指的是将VQ-VAE和GAN结合起来的模型。通过使用VQ-VAE的功能部分保持不变，同时使用检测器分辨decoder的输出和真实图像，从而提升生成图像的质量。

7 Image Formation Model

Occupancy and SDFs

本章节的目的是生成3D场景。表示方式有隐式和显式之分。其中，**显式表示 (Explicit Representations)** 直接使用几何数据结构（如网格、点云或体素）来表示3D场景。而**隐式表示 (Implicit Representations)** 则通过函数（如神经网络）来定义3D场景的形状和属性。

Signed/unsigned distance field (SDF, uSDF) 就是一种隐式表示方法。它通过一个函数来表示空间中每个点到最近表面的距离。对于SDF，距离值为正表示点在表面外部，负值表示点在表面内部。uSDF则只考虑距离的绝对值，不区分内部和外部。

而**Occupancy Field** 则是另一种隐式表示方法，它表示每个点是否被物体占据。通常1表示点在物体内部，0表示点在物体外部。

Neural Radiance Fields (NeRFs)

Radiance Field是描述在三维空间中任意一点往任意方向发出的光有多亮、什么颜色的一个函数。

$$\mathcal{L}(\theta) = \sum_i \|I_i - VolRender(\sigma_{\theta}, c_{\theta}, \Pi_i)\|^2$$

对于每个位置，都可以确定两个信息：密度 $\sigma_{\theta}(\mathbf{x})$ 和颜色 $c_{\theta}(\mathbf{x})$ 。与此同时还有一个投影函数 Π_i ，表示摄像机方位。通过体积渲染（Volume Rendering）技术，我们可以将这些信息整合起来，生成最终的图像 I_i 。

GSplats (Gaussian splats)

对于一个场景来说，简单使用NeRF的开销太大。所以我们将相近的点进行聚合，形成**高斯斑点 (Gaussian Splats)**。每个斑点是一个包含了坐标、大小、方向、密度和颜色的椭球（ellipsoid）。

$$\mathcal{L}(\theta) = \sum_i \|I_i - \text{SplatRender}(P_\theta, \Pi_i)\|^2$$

Structure from Motion (SfM)

以上两种模型的运算都可以通过**Structure from Motion (SfM)** 的方式训练。SfM通过分析一组图像中的特征点，估计摄像机的位置和姿态，从而重建三维场景的结构。也就是说，在这种情况下，对于每张照片 Π_i 是未知的，需要通过SfM来估计。

8 Self-supervised Learning

这个概念我们之前已经介绍过，就是指不需要标签的数据进行训练。

CLIP (Contrastive Language–Image Pre-training)

Augmentation 是指对输入数据进行各种变换（如旋转、裁剪、颜色调整等），以增加数据的多样性，从而提高模型的泛化能力。这当中可以将来自其他图片的处理结果作为**negative samples（负样本）**，防止模型觉得所有图片生成的空间都具有相同的语义。在负样本中，即使真的有语义相似的图片，一方面这是很偶然的，其次模型也会被迫区分它们，从而学习到更细粒度的特征。

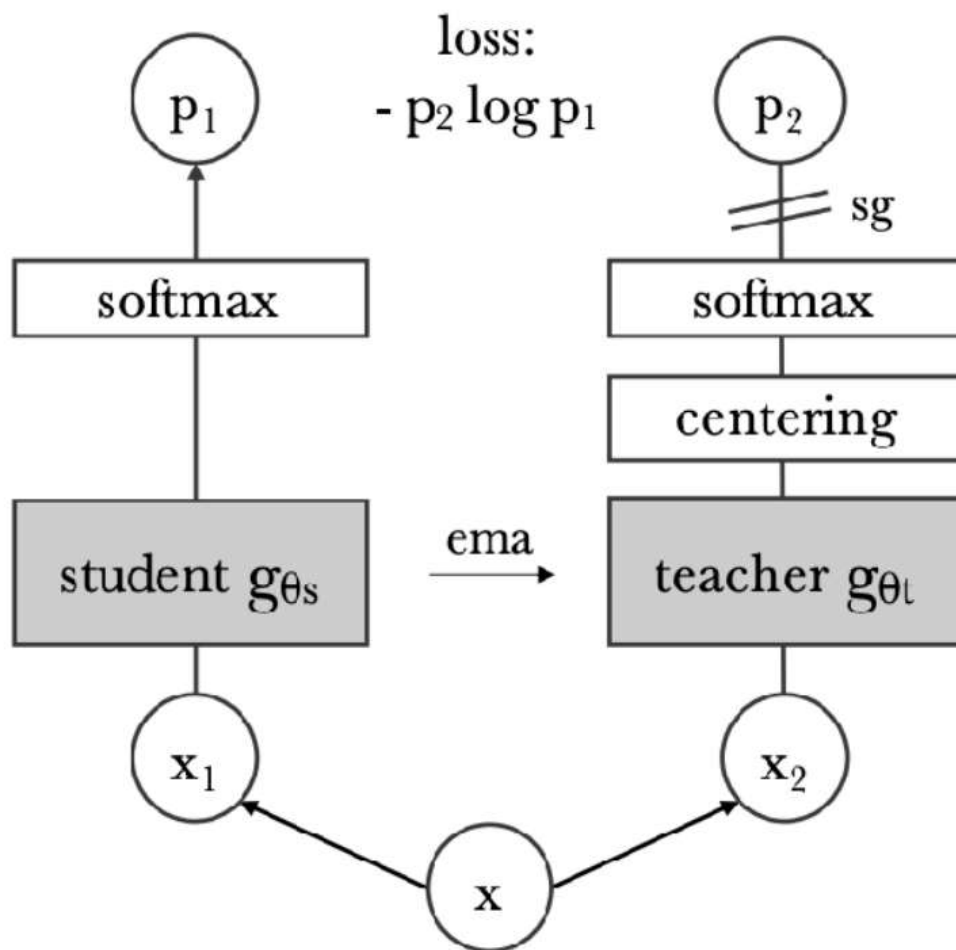
Pretext Tasks 是指在没有标签的数据上训练模型，通过设计一些辅助任务（例如遮挡一部分要求模型预测被遮挡的内容）来学习有用的特征表示。

embedding space（嵌入空间） 很类似于我们最开始说的用于分类图片的向量空间，同时通过上面的方法训练的模型可以将图片和文字映射到嵌入空间中。由于上述方法强制模型理解数据的结构，具有相似语义的内容往往会在嵌入空间中靠得更近，也就是说向量接近的图片和文字的语义更相似。

自监督模型在许多任务上可以 zero-shot 使用。但如果目标是特定任务，例如分类或语义分割，通常仍需少量有标签数据进行 fine-tuning 来提升性能。

Dino.v1

DINO (Self-Distillation with No Labels) 使用了两个网络：一个是学生网络（Student Network），另一个是教师网络（Teacher Network）。教师网络的参数是学生网络参数的指数移动平均（Exponential Moving Average, EMA）。目标是让学生网络的输出尽可能接近教师网络的输出。



其中 $\mathbf{sg}$ 表示stop gradient，也就是不对教师网络进行反向传播。

EMA 的公式为：

$$\theta_{teacher} = m \cdot \theta_{teacher} + (1 - m) \cdot \theta_{student}$$

其中 m 是一个接近1的动量参数。

centering 指的是对教师网络的输出的各个维度进行均值调整，以防止结果偏向某几个维度，使得模型彻底失效。

teacher 是 student 的历史平均版本，比 student 更稳定；

student 尝试模仿 teacher 的输出；

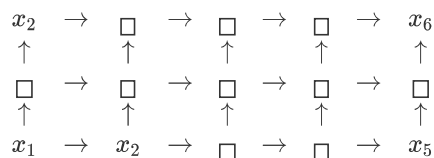
没有标签也能训练语义强的特征。

9 AutoRegressive and Diffusion Models

AutoRegressive (AR, 自回归)

自回归的意思是，依赖之前的一定数量的输出结果来预测下一个输出结果。例如GPT会根据前面的文字来判断下一个要回复的词是什么。在图像领域就是根据前面的像素来预测下一个像素。

这种结构很接近一个**RNN**



你可以看到左上角的输出只接受初始输入，而右下角的输出则接受了所有之前的输出。

当然，我们可以选择**CNN**，也就是增加一些隐藏层，而上面的神经元不是接受左侧的输入，而是接受多个下方的输入。这种情况下要用**causal convolution**，也就是卷积核只能覆盖左下方和下方的输入。

又或者使用**Transformer**，也就是每个神经元使用self-attention机制读取下方的所有输入。当然，也是**causal self-attention**，也就是只能读取正下方及其左侧的输入。

在图片生成时，自然也有**pixelRNN**（也可以使用之前提到的LSTM技术）和 **pixelCNN** 的选择，当然和transformer相比，它们分别有无法并行和无法读取远处数据的缺点。

conditioned synthesis

生成图像时可以使用**条件生成 (Conditioned Synthesis)**。例如语义，也就是包括提示词；或者深度，这样新生成的图片就具有大致相同的深度信息。

Diffusion models（扩散模型）

前向扩散 (Forward Diffusion) 是指将数据逐渐添加噪声的过程。假设我们有一个初始图像 x_0 ，我们可以通过以下公式逐步添加噪声：

$$x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_t$$

其中， ϵ_t 是从标准正态分布中采样的噪声。随着 t 的增加，图像逐渐变得模糊，最终接近纯噪声。

反向扩散 (Reverse Diffusion) 需要学习

$$f_{\theta}(x_t, t)$$

算出

$$x_{t-1} = x_t + f_{\theta}(x_t, t)$$

来逐步去除噪声，恢复原始图像。这个过程通常使用神经网络来预测每一步的噪声成分，从而实现从噪声到清晰图像的转换。

这样我们就知道了如何从噪声生成图像。注意了，我们使用大量的图片训练的这个模型，可以训练处一个可以去噪的模型，而它的出发点可以是**任何噪声图**。

Faster sampling: DDIM

DDIM (Denoising Diffusion Implicit Models) 是通过给出一个Ordinary differential equation（ODE，常微分方程）从 x_T 回到 x_0 的过程，从而实现更快的采样。

在这种情况下可以使用远比前面的方式少得多的步数。

Latent Diffusion Model

通过提前将图片压入潜空间，然后再加噪。在denoise的过程中，使用一个U-Net结构的神经网络来进行去噪。就可以大幅度减少计算量。与此同时，也可以更容易地在U-Net中使用cross-attention机制，从而实现条件生成，例如指定提示词等。